CMSC132: Threads

By Naren Ayinala

May 2026

Contents

1 What Is a Thread? **2**
1.1 Concurrency vs Parallelism 2

2 Creating Threads in Java **2**
2.1 Extending Thread 2
2.2 Implementing Runnable 3
2.3 Lambda Expressions 3

3 Thread Lifecycle and Common Methods **3**
3.1 Using join() to Wait for Completion 4
3.2 sleep() and InterruptedException 4

4 Shared Data and Race Conditions **4**

5 Critical Sections and synchronized **5**
5.1 Synchronized Blocks 5

6 Atomic Variables and volatile **5**

7 Deadlock and Lock Ordering **6**

8 wait(), notify(), and notifyAll() **6**

9 Executors and Thread Pools **6**

10 Thread-Safe Collections **7**

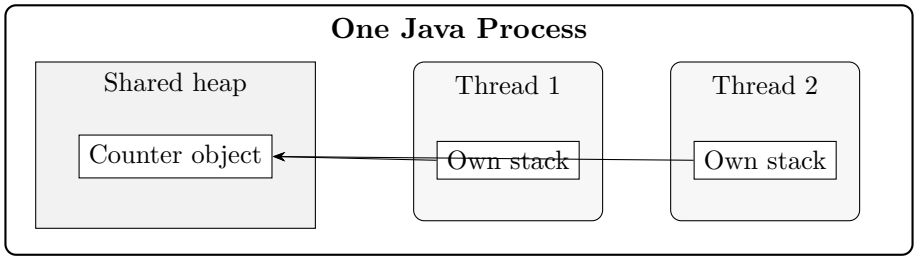
11 Practice and Common Mistakes **7**

12 Quick Reference **8**

1 What Is a Thread?

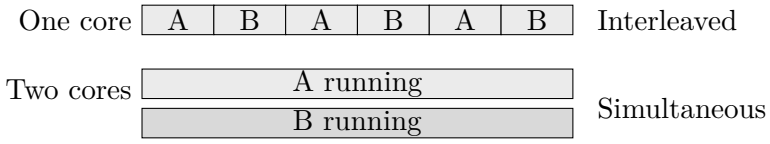
A **thread** is an independent path of execution inside a running program. Normally, when we write a Java program, statements in `main()` run one after another. With threads, multiple tasks can make progress during the same period of time. For example, a music player might play audio, update the progress bar, and respond to clicks without making the user wait for one task to finish before starting the next.

A **process** is a running program with its own memory space. Threads belonging to the same process share the heap (objects and static fields), but each thread has its own call stack and local variables. This shared memory is convenient, but it is also why threading introduces bugs that ordinary sequential code does not have.



1.1 Concurrency vs Parallelism

Concurrency means tasks overlap in progress: the CPU may switch between them. **Parallelism** means tasks actually execute simultaneously on different cores. You can have concurrency with just one CPU core, while parallel execution requires multiple processing resources.



2 Creating Threads in Java

There are two common introductory approaches: extending `Thread` and implementing `Runnable`. Both work, but `Runnable` is usually preferred because the task is separated from the thread object and Java classes can extend only one class.

2.1 Extending Thread

```
class MyThread extends Thread {
    @Override
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println("Worker: " + i);
        }
    }
}
public class Demo {
    public static void main(String[] args) {
        MyThread worker = new MyThread();
        worker.start(); // asks JVM to start a NEW thread
        System.out.println("Main continues");
    }
}
```

Remember: Calling `run()` directly is just an ordinary method call. Calling `start()` schedules a new thread, which then executes `run()`. The order of the printed lines is not guaranteed.

2.2 Implementing Runnable

```
class PrintTask implements Runnable {
    @Override
    public void run() {
        for (int i = 0; i < 3; i++) {
            System.out.println(Thread.currentThread().getName()
                               + " -> " + i);
        }
    }
}
public class Demo {
    public static void main(String[] args) {
        Thread a = new Thread(new PrintTask(), "Alpha");
        Thread b = new Thread(new PrintTask(), "Beta");
        a.start();
        b.start();
    }
}
```

Both threads run the same task definition, but each has its own execution. Their outputs can interleave differently every run.

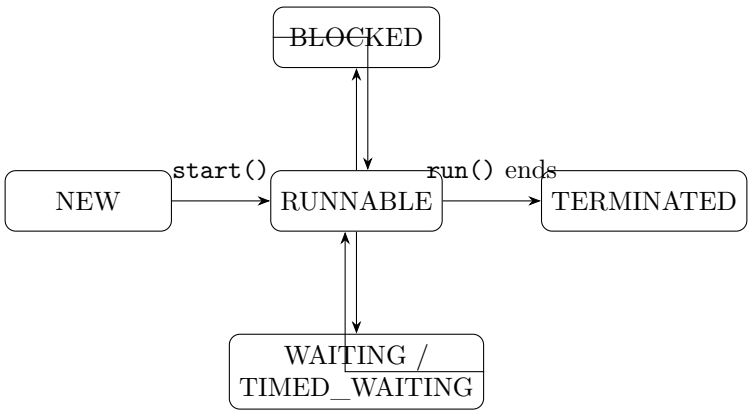
2.3 Lambda Expressions

Since `Runnable` is a functional interface (one abstract method), we can shorten it with a lambda expression:

```
Thread worker = new Thread(() -> {
    for (int i = 0; i < 5; i++) {
        System.out.println("Task " + i);
    }
});
worker.start();
```

3 Thread Lifecycle and Common Methods

A thread moves through states including `NEW`, `RUNNABLE`, `BLOCKED`, `WAITING`, `TIMED_WAITING`, and `TERMINATED`. In Java, `RUNNABLE` includes both ready-to-run and actively running; Java does not expose a separate `RUNNING` state in `Thread.State`.



`start()`	Start a new thread; may be called only once per `Thread` object.
`run()`	Task body; direct call does not start a new thread.
`sleep(ms)`	Pause current thread for approximately that long; does not release locks.
`join()`	Make the calling thread wait until another thread finishes.
`isAlive()`	True after start while the thread has not terminated.
`interrupt()`	Request interruption; does not forcibly stop the thread.

3.1 Using join() to Wait for Completion

```
public class JoinExample {
    public static void main(String[] args)
            throws InterruptedException {
        Thread worker = new Thread(() -> {
            System.out.println("Working...");
        });
        worker.start();
        worker.join(); // main waits for worker to finish
        System.out.println("Finished!");
    }
}
```

Without `join()`, `Finished!` could print before the worker's message. With `join()`, the worker completes first.

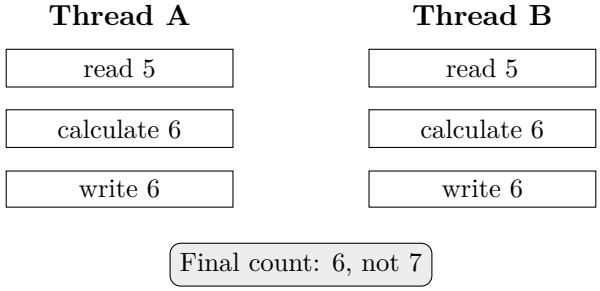
3.2 sleep() and InterruptedException

```
Thread timer = new Thread(() -> {
    try {
        Thread.sleep(500); // milliseconds
        System.out.println("Half a second later");
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
});
timer.start();
```

Remember: `sleep()` does not guarantee exact timing. Scheduling delays may make the pause longer. It also does not release a monitor lock held by the sleeping thread.

4 Shared Data and Race Conditions

Imagine two threads each executing `count++` one thousand times. We might expect the final count to be 2000, but that is not guaranteed. Why? Because `count++` is not one indivisible operation: it reads the old value, adds one, and writes the new value.



```
class Counter {
    private int count = 0;
    void increment() { count++; } // NOT thread-safe
    int getCount() { return count; }
}
```

This is a **race condition**: the result depends on which thread runs each step and when. Shared mutable data must be coordinated.

5 Critical Sections and synchronized

A **critical section** is code accessing shared mutable state that must not be executed concurrently without proper coordination. Java's `synchronized` uses a monitor lock so that only one thread at a time can hold that same lock.

```
class SafeCounter {
    private int count;
    public synchronized void increment() {
        count++; // guarded by this object's lock
    }
    public synchronized int getCount() {
        return count;
    }
}
```

For an instance synchronized method, the lock is `this`. For a static synchronized method, the lock is the corresponding `Class` object. Two different `SafeCounter` instances have different locks.

5.1 Synchronized Blocks

Sometimes you only want to protect a small portion of a method:

```
class BankAccount {
    private int balance = 100;
    public void withdraw(int amount) {
        synchronized (this) {
            if (balance >= amount) {
                balance -= amount;
            }
        }
    }
    public synchronized int getBalance() { return balance; }
}
```

Remember: Every access that needs protection must use the same coordination policy. Synchronizing only the writer while leaving an unsynchronized reader may still cause visibility or consistency problems.

6 Atomic Variables and volatile

`AtomicInteger` provides atomic operations such as `incrementAndGet()` without requiring us to write a synchronized increment method.

```
import java.util.concurrent.atomic.AtomicInteger;
class AtomicCounter {
    private final AtomicInteger count = new AtomicInteger();
    void increment() { count.incrementAndGet(); }
```

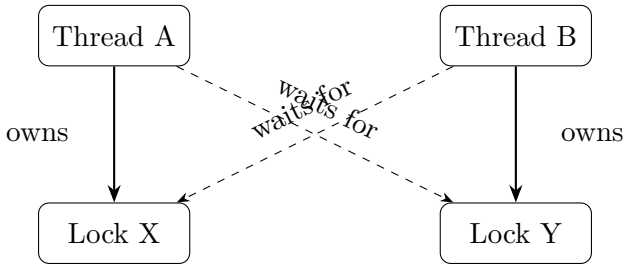
```
    int getCount() { return count.get(); }
}
```

The keyword `volatile` makes writes to a field visible to other threads and establishes ordering guarantees, but **does not make `count++` atomic**. It is suitable for simple status flags, not compound updates.

```
class Worker implements Runnable {
    private volatile boolean running = true;
    public void stop() { running = false; }
    @Override public void run() {
        while (running) { /* do small pieces of work */ }
    }
}
```

7 Deadlock and Lock Ordering

A **deadlock** occurs when threads wait for locks held by one another and neither can proceed. For example, A owns lock X and waits for Y, while B owns Y and waits for X.



One standard prevention technique is to always acquire multiple locks in a consistent order. Other tools include timeouts and avoiding nested locks when possible.

8 wait(), notify(), and notifyAll()

These are methods on `Object`, not `Thread`. A thread must own an object's monitor to call its `wait()`, `notify()`, or `notifyAll()`. Unlike `sleep()`, `wait()` releases that monitor while waiting.

```
class Signal {
    private boolean ready = false;
    synchronized void await() throws InterruptedException {
        while (!ready) { wait(); } // always recheck condition
    }
    synchronized void signal() {
        ready = true;
        notifyAll();
    }
}
```

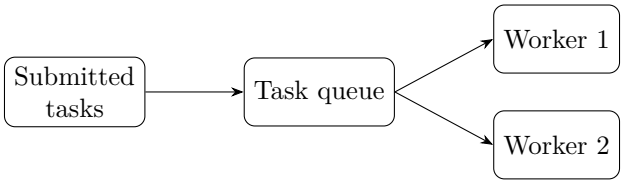
We use a `while` loop, not an `if`, because wakeups can occur even when the condition is still false. In larger applications, higher-level concurrency utilities are often easier to use correctly.

9 Executors and Thread Pools

Creating a brand-new thread for every tiny task has overhead. An `ExecutorService` manages a pool of worker threads and a queue of tasks.

```
import java.util.concurrent.*;
public class PoolDemo {
    public static void main(String[] args) throws Exception {
        ExecutorService pool = Executors.newFixedThreadPool(2);
        try {
            Future<Integer> result = pool.submit(() -> 7 * 6);
            System.out.println(result.get()); // 42
        } finally {
            pool.shutdown();
        }
    }
}
```

`Runnable` represents work with no return value; `Callable<T>` can return a value and throw checked exceptions. A `Future<T>` represents a result that may not be ready yet. Calling `get()` waits if necessary.



10 Thread-Safe Collections

Java also provides `ConcurrentHashMap`, `BlockingQueue`, and other utilities from `java.util.concurrent`. These are designed for common coordination patterns and are generally preferable to inventing your own locking logic.

```
import java.util.concurrent.*;
BlockingQueue<String> messages = new ArrayBlockingQueue<>(10);
// Producer thread:
messages.put("hello");
// Consumer thread:
String msg = messages.take();
```

`put()` can wait if the queue is full; `take()` can wait if it is empty. This makes a bounded producer–consumer system easier to implement.

11 Practice and Common Mistakes

1. **What is the difference between `start()` and `run()`?** `start()` begins a new thread; directly calling `run()` executes on the calling thread.
2. **Is `volatile int count` enough for two threads doing `count++`?** No. Visibility is not atomicity.
3. **What does `join()` do?** Makes the calling thread wait for the target thread to terminate.
4. **Does `sleep()` release synchronized locks?** No. `wait()` releases the monitor on which it is invoked.
5. **Why use a thread pool?** To reuse threads, control concurrency, and queue work.
6. **Can thread output order be predicted?** Usually not without explicit synchronization or ordering constraints.

12 Quick Reference

Concept	Key idea
`Thread`	Unit of execution in a process
`Runnable`	Task with `run()` and no return value
`synchronized`	Mutual exclusion plus memory visibility guarantees
`volatile`	Visibility/ordering for a field, not atomic compound operations
`AtomicInteger`	Atomic operations on an integer
`join()`	Wait for a thread to finish
`ExecutorService`	Run tasks using managed workers
Deadlock	Circular waiting for resources